# 編譯器 Chapter 1 濃縮筆記 — 導論

課程用書：Fischer · Cytron · LeBlanc《Crafting a Compiler》
整理 2026-09-23 | A4 列印版

## 1 | 編譯器是什麼？

**編譯器（compiler）**：一支「翻譯程式」的程式。輸入是**原始語言（source language）**寫成的程式文字，輸出是**目標語言（target language）**的程式文字——課程先假設目標是**組合語言（assembly language）**，之後再由**組譯器（assembler）**翻成 CPU 真正能執行的**機器語言（machine language）**。這整個翻譯過程叫 **compilation**。




- **使用者視角**會把上面收成一格：「程式語言進去、機器語言出來」，因為編譯＋組譯通常一個指令做完。但嚴格說編譯器只走到組合語言。
- 編譯器**翻譯程式，不負責執行**。執行是組譯、連結、載入之後的事（或交給直譯器，見 §4）。
- 目標語言不一定是組合語言：也可以是虛擬機指令（§3）或甚至 C（習題 11），都仍算編譯。

## 2 | 前端（Front-end）與後端（Back-end）




**IR（intermediate representation，中介表示）**：編譯器內部用來表示程式的資料結構，不是給人看的程式碼。前端把原始碼「分析」成 IR；後端把 IR「合成（synthesis）」成目標碼。

- **前端**依賴來源語言（文法、型別、語意規則）；**後端**依賴目標機器（暫存器、指令集、呼叫慣例）。
- **為什麼要這樣切？**換 CPU 只要重寫後端；換語言只要重寫前端。（補充：M 種語言 × N 種機器，只需寫 M 個前端＋N 個後端，而非 M×N 套完整編譯器。）
- 本課後半的 scanner、parser、type checker、translator 屬前端；optimizer 與 code generator 屬（中）後端。符號表跨兩端（§5）。

**易混**：這裡的 front / back 跟「使用者介面」無關——front 是**分析端**、back 是**生成端**。

## 3 | 三種目標碼：編譯器可以產出什麼？

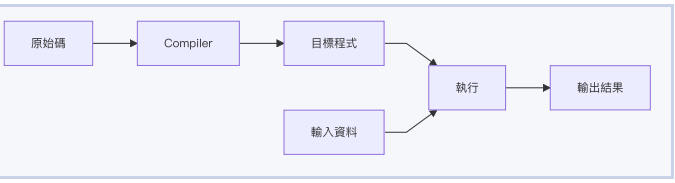
| 種類 | 產出內容 | 用在哪裡 |
|---|---|---|
| **Pure machine code**<br>純機器碼 | 只有該 CPU 的指令集，**不假設作業系統或函式庫存在**——沒有 read、沒有視窗 API、沒有 malloc，I/O 與記憶體管理全部自己用指令做 | 實作作業系統、開機載入程式、無 OS 的微控制器（所謂 system implementation languages） |
| **Augmented machine code**<br>擴充機器碼 | 同一個指令集，但編譯器可以額外發出「呼叫服務」的碼，服務分兩類：<br>① **OS routines**：system calls、window APIs（作業系統提供）<br>② **runtime support routines**：語言自帶的執行期支援（啟動碼、例外處理、垃圾回收） | 一般使用者空間的 C / C++ 程式——用到 `printf` 就是這類 |
| **Virtual machine code**<br>虛擬機碼 | 全部由**虛擬機（VM）**的指令組成（例：Java 的 **bytecode** 就是 JVM 的指令集）。任何裝了 **VM interpreter** 的電腦都能跑同一份 bytecode | Java 這類重視可攜性的語言 |

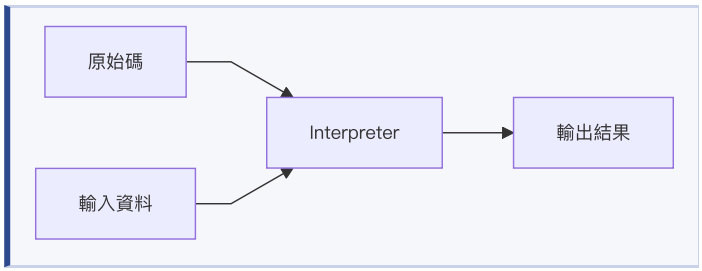
**虛擬機的可攜性從哪來？**兩句話一起讀：①「只需寫一份 VM interpreter」＝bytecode 格式與編譯器全世界只做一次；②「不同平台需要不同的 VM interpreter」＝每種真實 CPU / OS 還是各要一支能執行 VM 指令的程式。省下的是「每個平台各寫一套完整的 Java→native 後端」。這條路現在划算，是因為 CPU 夠快，多一層解譯的成本負擔得起。

**易混**：① augmented 不是第三種 CPU，是「同一指令集＋約定好可呼叫的例程」；OS 例程與語言 runtime 是兩類，不要混。② 有了 JVM interpreter **仍然需要 javac**：interpreter 吃的是 bytecode，不是原始碼。

## 4｜編譯器 vs 直譯器

**直譯器（interpreter）**：不產生獨立的目標程式，而是**直接執行原始碼**——邊讀程式的輸入資料、邊產出執行結果。它不是「沒有翻譯」，而是把翻譯攤到執行期間做。




| | 編譯器 | 直譯器 |
|---|---|---|
| 階段 | **compilation**（翻譯）與 **execution**（執行）兩段分開 | 一段：邊翻譯邊執行 |
| 輸入 | 只有原始碼 | 原始碼 ＋ 程式的輸入資料 |
| 輸出 | 另一支程式（之後可重複執行） | 該次執行的結果 |
| 時間觀 | 編譯一次、執行很多次 | 每次執行都從原始碼重新走 |

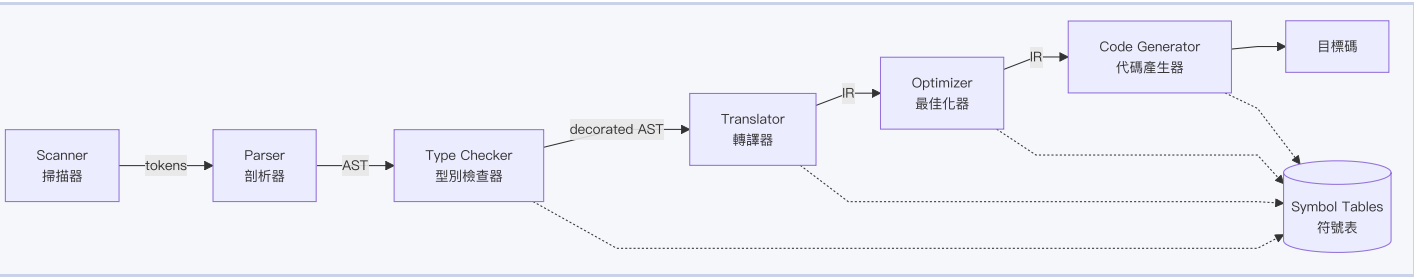
### 直譯器的三種執行策略

1. **直接執行原始碼**：最單純也最慢。例：LISP、早期 BASIC。
2. **先譯成較有效率的中介表示、立刻執行**：例：Perl、Python、MATLAB、Ruby。
3. **執行「直譯系統內建編譯器」事先編好的碼**：例：UCSD Pascal。

**Java 是 2＋3 的混合**：`javac` 先把原始碼編成可儲存的 bytecode（像策略 3 的「事先編好」），JVM 執行時拿到的是中介表示而非原始碼（像策略 2）。

> **易混**：語言名稱不能當實作分類——同一個語言可以同時存在編譯實作與直譯實作。

## 5｜編譯器內部管線：語法導向編譯器（syntax-directed compiler）




實線箭頭＝主鏈資料流（箭頭上是資料單位）；虛線＝對符號表的讀寫。Optimizer 另有「可整個繞過」的捷徑（不最佳化也能生碼）。粗分：Scanner→Translator 是前端（分析），Optimizer、Code Generator 是後端（合成）。

| 階段 | 吃 | 吐 | 在做什麼＋名詞解釋 |
|---|---|---|---|
| **Scanner**<br>掃描器 | 字元 | **Tokens** | 把字元聚成**詞法單位 token**：`foo_1` 是 1 個 identifier（不是 5 個字元）、`42` 是 1 個 integer。空白通常直接丟掉，不當 token。 |
| **Parser**<br>剖析器 | Tokens | **AST** | 依 **CFG（context-free grammar，上下文無關文法）**檢查結構，通常建出 **AST（abstract syntax tree，抽象語法樹）**——比 parse tree 精簡：丟掉括號、中間非終端等只為語法存在的節點，留下運算與結構。 |
| **Type Checker**<br>型別檢查器 | AST | **Decorated AST** | 又稱 semantic analyzer（語意分析器）。**Decorate（裝飾）**＝在原樹節點上附加型別、常數值、符號表條目等資訊，不是另做一棵樹。檢查型別正確性、reachability（有沒有走不到的碼）、termination、例外處理。 |
| **Translator**<br>轉譯器 | Decorated AST | **IR** | **語意檢查通過才翻譯**成 IR（例：bytecode、三地址碼）。要求是「意義正確」——IR 的執行效果對應來源語意，不求快。 |
| **Optimizer**<br>最佳化器 | IR | 較好的 IR | 改善效能，**語意必須保持不變**；實例見 §7。可以被繞過。 |
| **Code Generator**<br>代碼產生器 | IR（或直接 AST） | 目標碼 | 把 IR 對應成組合語言（或 VM 指令）。主鏈最後一格。 |
| **Symbol Tables**<br>符號表 | 被右四階段讀寫 | | 記錄每個名稱的型別、位址等資訊。圖上只有 Type Checker、Translator、Optimizer、Code Generator 有虛線連到它；（補充：許多實作在 Scanner 掃到識別字時就會建表，只是圖沒畫。） |

> **易混**：parser 接受只代表「合乎 CFG」，不代表型別正確。`int x = "hi";` 可能掃得過、parse 得過，到 type checker 才被擋下。

### Scanner 的 C 實作重點（課程補充碼，已驗證可編譯執行）

- `int c = getchar()`：刻意用 `int` 不用 `char`，因為要多留一個負值 `EOF` 表示「讀完」。若用有號 `char`，位元組 `0xFF` 會跟 EOF 撞在一起。
- 送進 `isspace()` / `isalpha()` 前一律先轉 `unsigned char`——負值直接傳入是**未定義行為**（可能查到陣列外面）。
- 詞素放 64 bytes 堆疊陣列，**寫入前**先檢查長度上限，超長就報錯離開，不會緩衝區溢位。
- 「向前看一個字元」靠變數 `c` 保存：每個分支結束時 `c` 已是下一個未處理字元，所以不需要 `ungetc`。
- 實測行為：`foo@` → 印 `IDENT foo` 後報 illegal character；`1foo` → 切成 `INT 1` 和 `IDENT foo`（這是詞法規則的結果，不是 parser 的問題）；連續 64 個字母 → 報 lexeme too long。

## 6｜Bootstrapping：編譯器編譯自己

**Bootstrapping（自我編譯）**：當「來源語言＝實作語言」，而且被編譯的原始碼就是這套編譯器本身，這個過程叫 bootstrapping。描述每個編譯器用 **T 圖三欄**：source language（編譯誰） / target language（產出什麼碼） / implementation language（編譯器本身用什麼跑）。

### 例 A：把 x86 上的 C 編譯器移植到 ARM

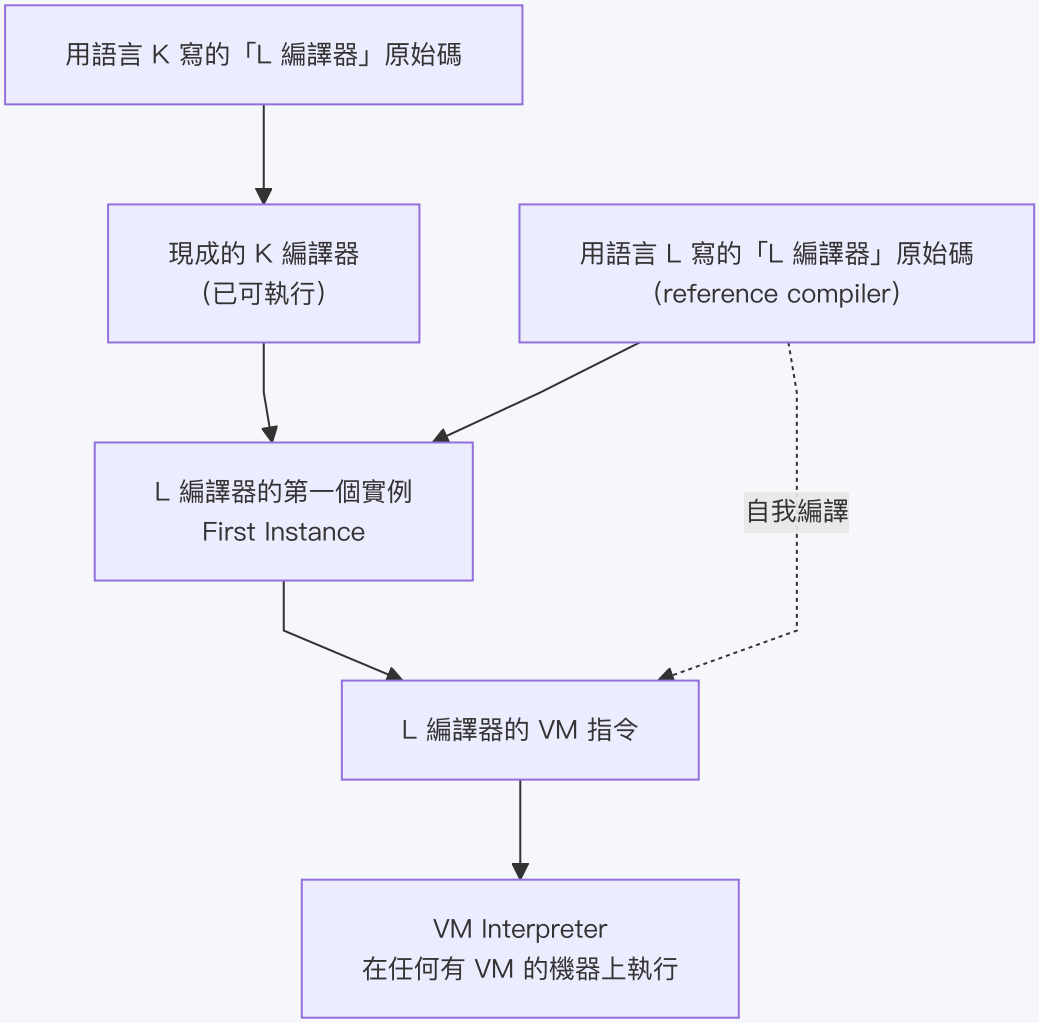
情境：手上有一個跑在 x86 的 C 編譯器 X，它是用 C 寫的（原始碼 P）。ARM 機器上還沒有任何 C 編譯器。做法：把 P 的 code generation 改成產生 ARM 組合語言，再「餵給自己」。

| 步驟 | 在哪台機器跑 | 輸入 | 產出（source / target / impl） |
|---|---|---|---|
| 1. 現況 | x86 | —（X 已是二元檔） | 可跑的 X（C / x86 / C） |
| 2. 改 P 的 code generator | 只改原始碼，還沒編譯 | P | P′：會發 ARM 碼的編譯器原始碼（C / ARM / C，**還不能跑**） |
| 3. 把 P′ 餵給 X（＝「餵給自己」） | x86 | P′ | X′：**cross-compiler**（在 x86 上跑、產出 ARM 碼）（C / ARM / **x86**） |
| 4. 再把 P′ 餵給 X′ | x86 | P′ | 能在 ARM 上跑的 C 編譯器（C / ARM / **ARM**） |

補充 **cross-compiler（交叉編譯器）**：在 A 機器上跑、卻產生 B 機器碼的編譯器。若目標是「ARM 上能跑的編譯器」，只到第 3 步是不夠的——X′ 仍是 x86 二元檔，必須做第 4 步。

**易混**：只改了 P 的 code generator、還沒重新編譯前，正在跑的 X 發出的仍是 **x86** 碼——跑的是舊二元檔。

### 例 B：借別的語言跨過第一次（VM 路線）

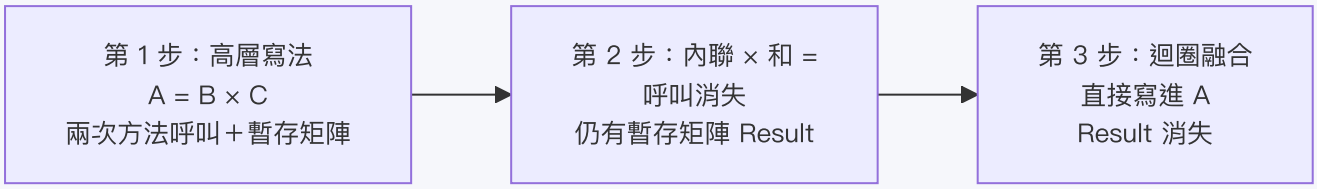



1. 先用**已存在的 K 編譯器**，把「用 K 寫的 L 編譯器」編出來 → 得到 L 的第一個可執行編譯器。這一步跨過「雞生蛋」問題。
2. 再用它編譯「用 L 自己寫的 reference compiler」→ 得到 L 編譯器的 VM 指令。
3. VM 指令交給 VM interpreter 執行——「VM 指令＋interpreter」合起來就是一個**可攜的 L 編譯器**，任何支援該 VM 的架構都能跑，而且它能編譯自己。

**K 什麼時候退場？**做出第一個實例之後就不需要了；之後只用 L 自己的編譯器與 VM interpreter。這正是「Chicken & Egg」漫畫的答案：第一個編譯器用另一種已存在的語言寫（或先做極小的手寫翻譯器再逐步放大）。

## 7｜Optimizer 實例：矩陣乘法 A = B × C 的三步改造

名詞先備：**過載運算子（overloaded operator）**＝讓 ×、= 這種符號對自訂型別（矩陣）有意義，底層其實是函式呼叫。**內聯（inlining）**＝把函式本體展開貼到呼叫點，省掉呼叫開銷。**迴圈融合（loop fusion）**＝把兩個跑同樣範圍的迴圈合併成一個。**別名（alias）**＝兩個名字指到同一塊記憶體。




### 第 1 步：原本的樣子（工作藏在過載運算子裡）

```
A = B × C                      // 高層一行，實際呼叫兩個方法

×(Y, Z)：if Y.cols ≠ Z.rows then 丟例外      ← ① 先查維度
         for i, for j：Result[i,j] ← 0
             for k：Result[i,j] += Y[i,k] × Z[k,j]
         return Result                        // 多一塊暫時矩陣

=(To, From)：if 維度不合 then 丟例外          ← ② 再逐元素拷貝
             for i, for j：To[i,j] ← From[i,j]
```

代價：兩次方法呼叫開銷＋一塊額外的 Result 儲存。

### 第 2 步：內聯 ①②（呼叫消失）

```
for i = 1..N, for j = 1..N          // 乘法巢
    Result[i,j] ← 0
    for k = 1..N：Result[i,j] += B[i,k] × C[k,j]
for i = 1..N, for j = 1..N          // 拷貝巢（還在！）
    A[i,j] ← Result[i,j]
```

維度檢查在展開圖裡沒畫出來，不代表被刪掉——內聯討論的是熱迴圈。

### 第 3 步：融合兩個迴圈巢（Result 消失）

```
for i = 1..N
    for j = 1..N
        A[i,j] ← 0
        for k = 1..N
            A[i,j] += B[i,k] × C[k,j]     // 直接寫進 A
```

- 被融合的是「乘法巢」與「拷貝巢」**這兩組**都跑 i、j 的迴圈——**不是**把 i、j 併成單層；i、j、k 三層都還在。
- Result 沒了；**j 迴圈還在**，少掉的是第二組 i、j 拷貝迴圈。

**易混**：這是編譯器對 IR 做的迴圈融合與暫存矩陣消除，**沒有改矩陣乘法的數學定義**。前提是 A、B、C 互為不同物件；若有別名（例如 `A = A * C`），直接寫 A 可能蓋掉還沒讀完的值。

## 8｜HW 標準解答（習題 4・8・11）

### HW 習題 4：編譯期錯誤 vs 執行期錯誤

> 名詞：**compile-time error（編譯期錯誤）**＝翻譯階段就能發現的錯，整支程式直接不合法；**run-time error（執行期錯誤）**＝實際執行到那一行才爆的錯。對比例子：`a = b/0;`（除數是常數 0，編譯期就看得出）vs `c = 0; a = b/c;`（要到執行才發現）。

**題目**：該不該讓「含編譯期錯誤的程式」照樣執行、只在真的跑到非法碼時才停？讓程式因為「可能執行到」的非法片段整個不合法，有沒有好處？

> 教材解答 ① Compile-time error is a better approach.
> ② A program with compiler-time errors SHOULD NOT be allowed to execute.
> ③ The advantage is that it ensures all the source codes are syntactically correct.

補充 解答③把好處說成「語法正確」，但除零其實比較接近語意 / 執行期約束，不一定是 CFG 語法錯。另外注意：題目說 `b/0` 會被標成非法是**題目設定**——C 語言裡整數除零是未定義行為，clang 預設只給 warning 照樣產生執行檔；Java 也要加 `-Xlint:divzero` 才警告，執行期才丟 `ArithmeticException`。不要用工具的警告政策改寫教材立場。

### HW 習題 8：為什麼做 JVM interpreter 比較容易？

> 背景：C / C++ 通常直接編成真實 CPU 的機器碼（Intel x86、Sparc）；Java 編成 JVM 的機器碼（bytecode），而 JVM 不是真的處理器——它是在既有處理器上被解譯執行的，所以平台獨立。題中的「完整 Java compiler」指「直接對真實 CPU 生碼」的那種，不是 javac。

**題目**：為什麼為 JVM 做 interpreter 比做完整的 Java compiler 更容易、更快？虛擬機途徑的缺點是什麼？

> 教材解答 Building an interpreter for a virtual machine is easier than building a complete Java compiler is that there are **fewer phases** in developing interpreters. The disadvantages of this virtual machine approach is that the source code must be compiled to byte code, and then a java interpreter can execute it. **This takes more time**.

補充 「少的 phases」對照 §5 管線：完整 native 後端還要做目標指令選擇、暫存器配置、呼叫慣例；VM interpreter 只需實作一組規則的虛擬指令。時間代價＝多一層 bytecode 解譯，跟 §3「CPU 夠快才划算」同一件事。

### HW 習題 11：把 C 當「萬用組合語言」

> 名詞：**universal assembly language**＝C 的綽號，因為幾乎每種架構上都有高效率的 C 編譯器。於是有些編譯器乾脆**輸出 C 程式碼**當目標語言，再交給各平台的 C 編譯器走完最後一哩。

**題目**：編譯器輸出 C（而非特定機器語言）的優點與缺點？

> 教材解答 Generating C code is **more portable** than generating an assembly code for a particular machine language. However, the source code will be translated into C code, and then to assembly code. This is **time-consuming**.

補充 可攜性來自「每台機器早就有 C 編譯器」；代價是多一次 C→assembly 的翻譯，而且 C 的語意（整數寬度、未定義行為、呼叫慣例）會變成你的 IR 必須遵守的邊界。注意：這是把 C 當**目標語言**，跟本課作業「用 C 寫編譯器本身」是兩回事。

## 9｜自我檢查速答（考前掃一遍）

**Q：編譯器輸出 bytecode，還算編譯器嗎？**
A：算。廣義上編譯器＝「翻譯到較低階、不再當原始碼改」的翻譯器；目標是 VM 指令或 C 都可以。

**Q：使用者視角圖（語言→編譯器→機器碼）看得出前端 / 後端分界嗎？**
A：不能，它把編譯器當不透明方塊，還順手合併了組譯器。

**Q：只換 CPU、語言不變，要重寫哪一端？**
A：後端（code generator＋部分 runtime）；前端與 IR 留用。

**Q：管線圖上哪幾個方塊有虛線連到符號表？**
A：Type Checker、Translator、Optimizer、Code Generator 四個；Scanner、Parser 沒有。

**Q：編譯器的「執行階段」跑的是 source 還是 target？**
A：target。source 只在 compilation phase 出現。

**Q：`foo1` 是幾個 token？`1foo` 呢？**
A：`foo1` ＝1 個 identifier；`1foo` ＝ `INT 1` ＋ `IDENT foo` 兩個 token（詞法規則問題）。

**Q：迴圈融合後還有 Result 嗎？還有 j 迴圈嗎？**
A：Result 沒了；j 還在，少掉的是第二組 i、j 拷貝迴圈。

**Q：VM 路線 bootstrapping 中，哪一步之後就不再需要語言 K 的編譯器？**
A：做出「L 的第一個實例」之後；之後只用 L 自己的編譯器與 VM interpreter。

## 10｜名詞速查表

| 名詞 | 一句話解釋 | 名詞 | 一句話解釋 |
|---|---|---|---|
| compiler 編譯器 | 把程式從來源語言翻成目標語言的程式 | interpreter 直譯器 | 不產出目標程式，直接執行原始碼 |
| assembler 組譯器 | 把組合語言翻成機器碼 | compilation | 「原始碼→目標碼」的翻譯過程 |
| front-end 前端 | 分析原始碼、建出 IR 的那半 | back-end 後端 | 把 IR 合成成目標碼的那半 |
| IR 中介表示 | 編譯器內部的程式表示，不給人看 | symbol table 符號表 | 記錄名稱的型別、位址等資訊的表 |
| token | 詞法單位（identifier、integer…） | CFG | 上下文無關文法，描述語法結構 |
| AST 抽象語法樹 | 去掉括號等語法雜物後的程式樹 | decorated AST | 節點附上型別等語意資訊的 AST |
| pure machine code | 不假設 OS / 函式庫存在的純機器碼 | augmented machine code | 機器碼＋約定可呼叫的 OS / runtime 例程 |
| VM / bytecode | 虛擬機 / 它的指令集（如 JVM bytecode） | runtime support | 語言自帶的執行期支援（啟動碼、GC…） |
| bootstrapping | 編譯器編譯「自己」的原始碼 | cross-compiler | 在 A 機跑、產 B 機碼的編譯器 |
| inlining 內聯 | 把函式本體展開到呼叫點 | loop fusion 迴圈融合 | 合併範圍相同的迴圈，省暫存 |
| compile-time error | 編譯期即可發現的錯誤 | run-time error | 執行到該行才出現的錯誤 |
| undefined behavior | 語言不定義後果的行為（如 C 除零） | universal assembly language | C 的綽號：到處都有它的編譯器 |